

Pipeline Konzept

Alle Komponenten sind noch nicht in einem einheitlichen Scheduler eingebunden

Kontextvalidierung

- Abgleichen der OpenAPI Struktur -> Logging von konkreten Änderungen
- Abgleichen des API-Schlüssel mit dem der Website -> Logging von Änderungen
- Abgleichen der Nutzungsbedingungen -> Logging falls Änderung

API-Request und Rohdaten-Speicherung

- Query von Vorgängen
 - Initial: nach Vorgängen des letzten Monats
 - Scheduler: seit letzter Abfrage (15 Minuten Buffer einplanen)
 - Themenvvalidierung: Ist Vorgang Rente-bezogen? Mit Gemini wenn key supplied, sonst über keyword suche
 - Relevante Vorgänge in SQLite speichern
- Für jeden relevanten Vorgang:
 - Query für Vorgangspositionen
 - Query für Drucksachen
 - Query für Plenarprotokolle
 - Jeweils direkt in SQLite speichern und mit gespeicherten Entitäten und Attributen verknüpfen
 - Scheduler: seit letzter Abfrage (15 Minuten Buffer einplanen)

DBT Processing

- Textbereinigung (Markdown-Identifizier etc)
- Typ-/Vollständigkeits-/Eindeutigkeitschecks
- Datenminimierung mit MART-Modellen
- ID-based matching

Deliverable

- Erstellung einer Markdown Datei mit: Metrik zur Anzahl der relevanten Vorgänge der letzten Tage, mit Status Auskunft

Key Komponenten im Repository:

- **dip_pipeline.ipynb**
- **procedure_measure_report.py**
- **Dbt models (/models)**

Brainstorming notes

Jeder Schritt:

- Gedanken
- Worauf achten
- Welche Tools

An welcher Stelle KI? An welcher Stelle deterministik?

Beispiel API DIP:

Konzept:

Vogang:

- Hat mehrere Vorgangspositionen
- Hat mehrere Drucksachen
 - Haben Personen
- Hat Plenarprotokoll
 - Hat Aktivitäten
 - Haben Personen

Dabei:

- Nicht zu viele Daten - nur wichtige (Umso mehr Daten umso höher der Wartungsaufwand und die Fehleranfälligkeit)

Notwendige Datenbereinigung:

- Text bereinigung von markdown identifizern (e. G. /n)
- Matching (Falls abkürzung oder Titel nicht direkt Matched)
- Test ob Typ-Bedingungen stimmen? (Vollständig, richtiger Datentyp, eindeutig?)

Zusätzlicher Check für API Key:

- Auf website ein wort mit genau 42 Zeichen und mehr als 2 Großbuchstaben und mehr als 2 Ziffern hat als <p> property (Wahrscheinlichkeit das Wort mit den Characteristics existiert ist >0,000%) -> Else warne das API Key geändert wurde/Evtl LLM html / p checken lassen

Zusätzlicher Check für Endpoints: (Check veränderung in openapi yaml (Deterministisch, da syntax)

- Gibt es neue Endpoints?
- Hat sich ein Schema verändert?

DIP Nutzungsbedingungen:

- Am Ende der Ausgabe DIP als Quelle erwähnen
- Deterministisch -> Haben sich die Nutzungsbedingungen geändert?

Gesamter Vorgang:

Daten mit Jupyter ziehen, in SQLite speichern und mit DBT testen und verarbeiten.